

Глубокое обучение

Компьютерное зрение и NLP

Методы искусственного интеллекта

Лекция 12

Сверточные нейронные сети

Сверточные нейронные сети (Convolutional Neural Network, ConvNet, CNN) — искусственные нейронные сети, имеющие один или несколько сверточных слоев.

Сверточные слои позволяют моделям глубокого обучения эффективно обрабатывать пространственные структуры, что делает их особенно эффективными в приложениях компьютерного зрения.

Сверточные сети позволяют анализировать изображения в виде матрицы для поиска более сложных признаков в двумерном пространстве.

Также сверточные сети позволяют снизить вычислительные затраты на анализ больших изображений по сравнению с полносвязанными слоями.

Принцип работы сверточной нейронной сети напоминает принцип работы органов зрения: от простых признаков к более сложным.

Сверточные слои

Сверточные слои содержат фильтры (ядра).

Фильтр – небольшое окно, через которое сверточный слой «смотрит» на изображение.

Фильтр применяется от левого верхнего до правого нижнего угла изображения.

В результате применения фильтров сверточный слой сворачивает изображение.

Сверточный слой имеет меньшее количество параметров, чем полносвязанный слой.

Результатом сверточного слоя является двумерный массив.



Свойства фильтра

Каждый фильтр состоит из весов и смещения, значения которых корректируются в процессе обратного распространения ошибки.

Обычно используют следующие размеры фильтра: 3x3 и 5x5 пикселей.

Для изображений с 1 каналом (градации серого) фильтр 3x3 будет содержать $3 * 3 * 1 + 1 = 10$ параметров.

Для изображений с 3 каналами (RGB) фильтр 3x3 будет содержать $3 * 3 * 3 + 1 = 28$ параметров.

$$f(x) = w \cdot x + b = .01*.53+.09*.34+.22*.06+ \\ \dots+.13*.62-.69*.91+1.02*.34 + .20 = \\ -0.3917 + 0.20 = -0.1917 \\ y=\text{ReLU}(f(x)) \text{ или } y=\text{tanh}(f(x))$$

.01	.09	.22
-1.36	.34	-1.59
.13	-.69	1.02

веса ядра

•

.53	.34	.06
.37	.82	.01
.62	.91	.34

пиксели на входе

В сверточных слоях отсутствуют веса для каждого входного значения, но используется фильтр.

Весь фильтра НЕ меняются в процессе свертки и остаются общими для всех входных значений.

Множество фильтров

Обычно в сверточном слое используется множество фильтров.

Каждый фильтр позволяет выявлять уникальные признаки на отдельном слое (по аналогии с органами зрения).

Каждый фильтр генерирует высокое значение функции активации при обнаружении соответствующего признака: указывают, где на изображении находится соответствующий признак.

Результат фильтра обычно называют картой активации.

Следующие сверточные слои получают на вход карты активации и реагируют на более сложные признаки.

Более глубокие сверточные слои способны выделять более сложные объекты на изображениях.

Количество фильтров

Количество фильтров является гиперпараметром:

- Большое количество фильтров упрощает идентификацию сложных признаков. Необходимо учитывать сложность данных в решаемой задаче. Рекомендуемые значения: 8, 16, 32, 64, 128, 256, 512. Обычно начинают с 32 фильтров.
- Чем больше фильтров, тем выше вычислительная сложность при обучении и выводе.
- Если в сети несколько сверточных слоев, оптимальное количество фильтров для каждого слоя может значительно отличаться. Обычно на следующем слое количество фильтров увеличивается в два раза по сравнению с предыдущим: 32, 64, 128, 256, 512.
- Лучше использовать минимальное количество ядер, обеспечивающих минимальную стоимость на проверочных данных.
- Если удвоение количества ядер в слое снижает стоимость на проверочных данных, используйте большее значение.
- Если уменьшение числа ядер в слое не увеличивает стоимость на проверочных данных, используйте меньшее значение.

Пример работы сверточной сети

В примере сворачивается изображение размером 3x3 пиксела с 3 каналами (RGB).

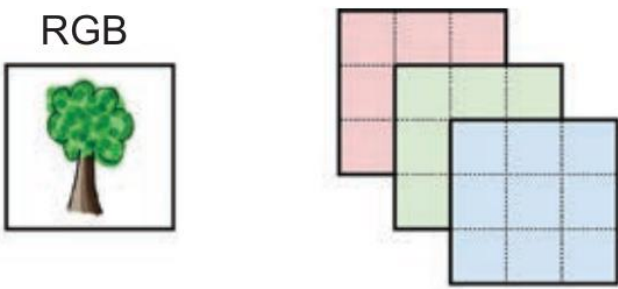
Размер изображения (3,3,3).

Размер фильтра 3x3.

Размер матрицы весов (3,3,3).

Смещение = 0.2.

2.64 – полученное значение в карте активации.



Изображения:

Красный	Зеленый	Синий
0.00 0.00 0.00 0.00 0.00	0.00 0.00 0.00 0.00 0.00	0.00 0.00 0.00 0.00 0.00
0.00 0.20 0.40 0.30 0.00	0.00 0.60 0.60 0.90 0.00	0.00 0.80 0.40 0.90 0.00
0.00 0.30 0.90 0.60 0.00	0.00 0.40 0.70 0.40 0.00	0.00 0.30 0.10 0.60 0.00
0.00 0.90 0.10 0.20 0.00	0.00 0.70 0.50 0.30 0.00	0.00 0.80 0.60 0.40 0.00
0.00 0.00 0.00 0.00 0.00	0.00 0.00 0.00 0.00 0.00	0.00 0.00 0.00 0.00 0.00

Веса:

0.10 0.20 0.60	0.60 0.60 0.90	0.80 0.40 0.90
0.60 0.80 0.70	0.40 0.70 0.40	0.30 0.10 0.60
0.50 0.40 0.30	0.70 0.50 0.30	0.80 0.60 0.40

Смещение: 0.20

значение z: $\sum (\begin{matrix} 0.00 & 0.00 & 0.00 \\ 0.00 & 0.16 & 0.28 \\ 0.00 & 0.12 & 0.27 \end{matrix} + \begin{matrix} 0.00 & 0.00 & 0.00 \\ 0.00 & 0.42 & 0.24 \\ 0.00 & 0.20 & 0.21 \end{matrix} + \begin{matrix} 0.00 & 0.00 & 0.00 \\ 0.00 & 0.08 & 0.24 \\ 0.00 & 0.18 & 0.04 \end{matrix}) + 0.2 = \begin{matrix} 2.64 & & \\ & & \\ & & \end{matrix}$

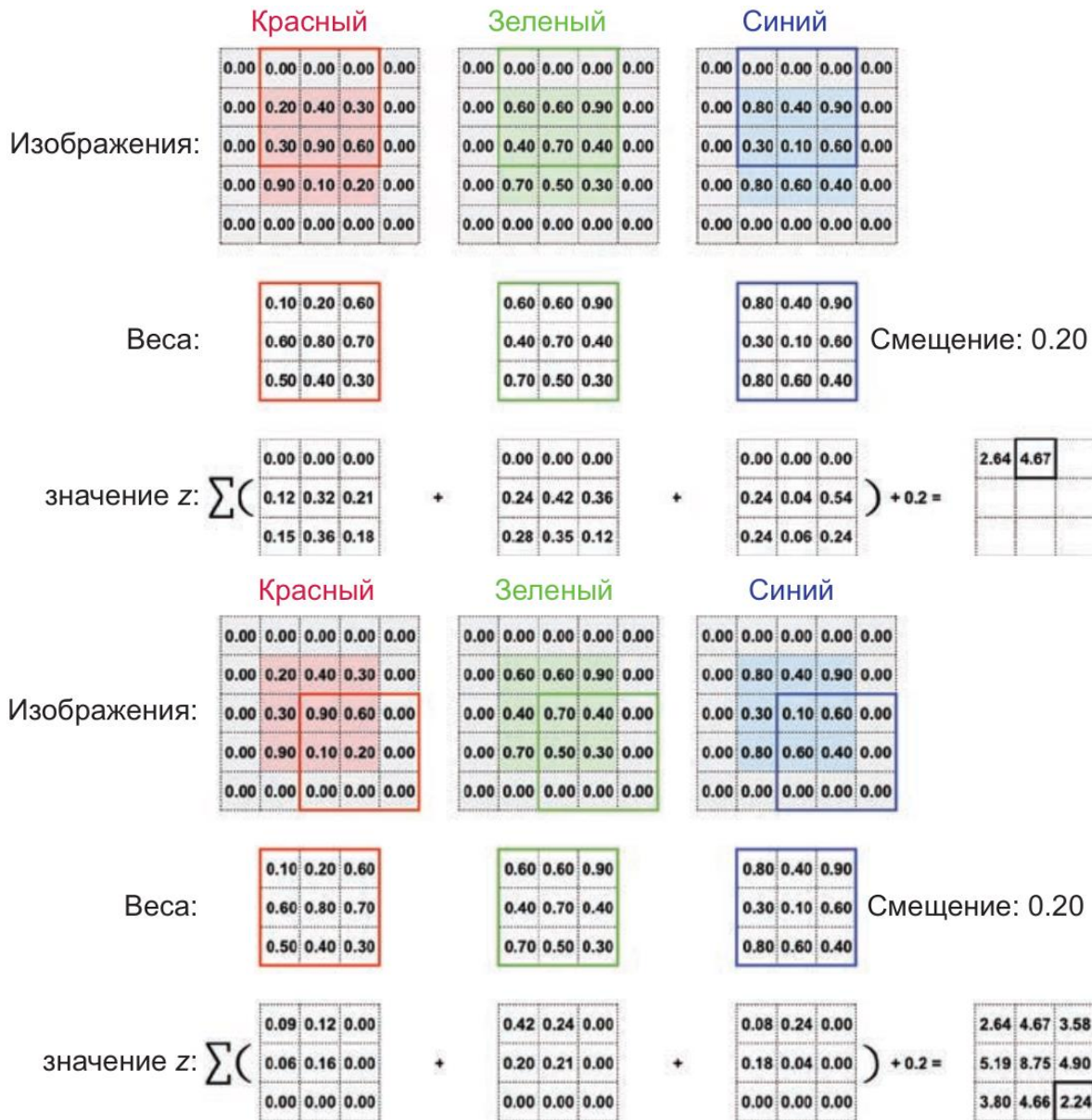
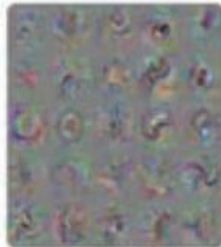
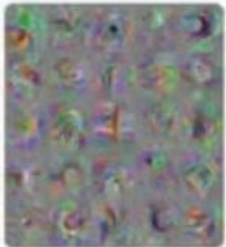
Еще пример работы сверточной сети

Карта активации имеет третье измерение (глубину), так как сверточный слой содержит несколько фильтров.

Для 16 фильтров карта активации будет иметь размер (16, 3, 3).

Каждый фильтр формирует свой слой карты активации (срез).

Каждый срез карты активации представляет отдельный признак в зависимости от роли фильтра.



Изображения:

Веса:

0.10	0.20	0.60
0.60	0.80	0.70
0.50	0.40	0.30

Смещение: 0.20

значение z:

0.00	0.00	0.00
0.12	0.32	0.21
0.15	0.36	0.18

+

0.00	0.00	0.00
0.24	0.42	0.36
0.28	0.35	0.12

+

0.00	0.00	0.00
0.24	0.04	0.54
0.24	0.06	0.24

) + 0.2 =

2.64	4.67	

Красный

0.00	0.00	0.00	0.00	0.00
0.00	0.20	0.40	0.30	0.00
0.00	0.30	0.90	0.60	0.00
0.00	0.90	0.10	0.20	0.00
0.00	0.00	0.00	0.00	0.00

Зеленый

Синий

Изображения:

Веса:

0.10	0.20	0.60
0.60	0.80	0.70
0.50	0.40	0.30

Смещение: 0.20

значение z:

0.09	0.12	0.00
0.06	0.16	0.00
0.00	0.00	0.00

+

0.42	0.24	0.00
0.20	0.21	0.00
0.00	0.00	0.00

+

0.08	0.24	0.00
0.18	0.04	0.00
0.00	0.00	0.00

) + 0.2 =

2.64	4.67	3.58
5.19	8.75	4.90
3.80	4.66	2.24

Свойства сверточных слоев

1. Сверточные слои позволяют моделям глубокого обучения учиться выявлять разные признаки в той или иной позиции.
2. Каждый фильтр может выявлять свой уникальный признак в любом месте во входных данных.
3. Сверточные сети сохраняют двумерную структуру изображений, позволяя идентифицировать признаки в пространстве.
4. Сверточные сети значительно сокращают количество параметров, необходимых для **моделирования** изображений, обеспечивая снижение затрат вычислительных ресурсов.
5. Сверточные сети обеспечивают более точное решение задач компьютерного зрения.

Гиперпараметры сверточных сетей

1. Количество фильтров в слое.
2. Размер фильтра.
3. Длина шага фильтра.
4. Дополнение (padding).

Количество фильтров

Количество фильтров является гиперпараметром:

- Большое количество фильтров упрощает идентификацию сложных признаков. Необходимо учитывать сложность данных в решаемой задаче. Рекомендуемые значения: 8, 16, 32, 64, 128, 256, 512. Обычно начинают с 32 фильтров.
- Чем больше фильтров, тем выше вычислительная сложность при обучении и выводе.
- Если в сети несколько сверточных слоев, оптимальное количество фильтров для каждого слоя может значительно отличаться. Обычно на следующем слое количество фильтров увеличивается в два раза по сравнению с предыдущим: 32, 64, 128, 256, 512.
- Лучше использовать минимальное количество ядер, обеспечивающих минимальную стоимость на проверочных данных.
- Если удвоение количества ядер в слое снижает стоимость на проверочных данных, используйте большее значение.
- Если уменьшение числа ядер в слое не увеличивает стоимость на проверочных данных, используйте меньшее значение.

Размер фильтра (ядра)

Типовые размеры фильтра:

1. 3x3 – обеспечивает высокую эффективность в широком перечне задач компьютерного зрения.
2. 5x5 – также часто используется.
3. 7x7 – максимальное значение фильтра, используется реже.

Чем больше размер фильтра, тем больше признаков будет учтено при свертке и модели будет сложнее обучиться.

Чем меньше размер фильтра, тем сложнее модели выявить значимые признаки.

Длина шага фильтра

Длина шага фильтра определяет количество пикселей для перемещения фильтра по изображению.

Типовые значения:

1. 1 пиксел. Часто применяется на практике.
2. 2 пиксела. Также часто используется на практике.
3. 3 пиксела. Используется реже, не рекомендуется.

Чем больше длина шага, тем больше шанс пропустить области изображения, имеющие значение для модели.

Увеличение шага увеличивает скорость свертки, так как требуется меньше вычислений.

Дополнение (padding)

Дополнение позволяет «окружить» изображение нулями для получения размера карты активации равного размеру изображения.

Значения данного гиперпараметра должно гарантировать получение действительного размера для карты активаций (целочисленного значения).

Слои субдискретизации

Слои субдискретизации (pooling layers) – слои для уменьшения общего числа параметров сверточной сети и снижения ее сложности.

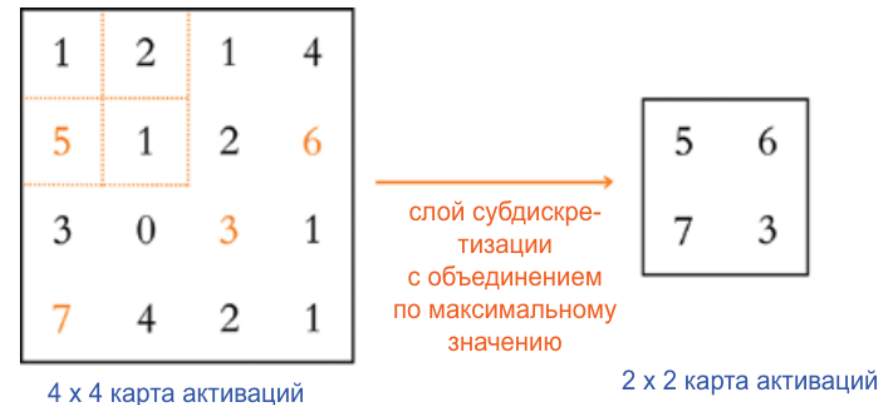
Позволяют ускорить вычисления и избежать переобучения.

Слой субдискретизации уменьшает размер карт дискретизации без изменения количества их срезов (глубины).

Принцип работы слоя субдискретизации аналогичен сверточному слою: слой скользит по карте активации для сокращения данных.

Гиперпараметры:

1. Размер фильтра (2x2).
2. Длина шага фильтра (2).
3. Функция агрегации (**max**, avg, L2).



Применение компьютерного зрения

Классификация:

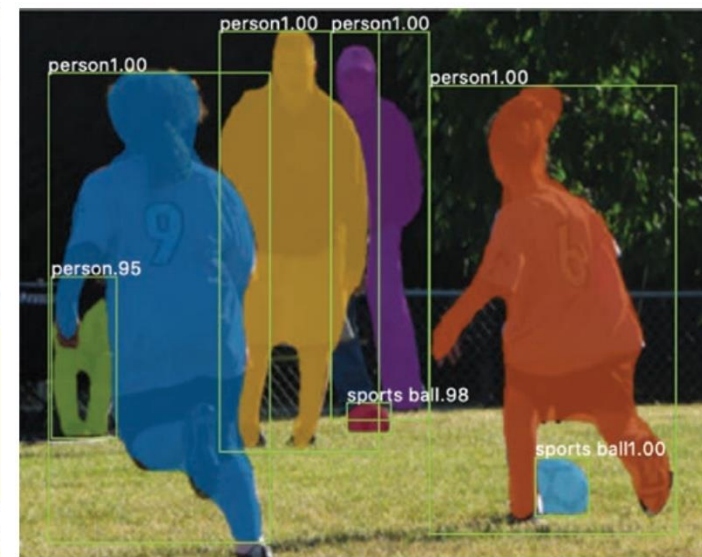
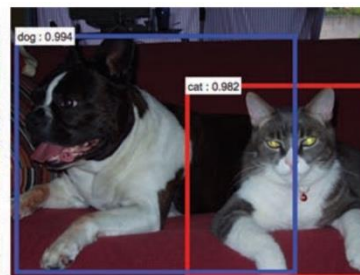
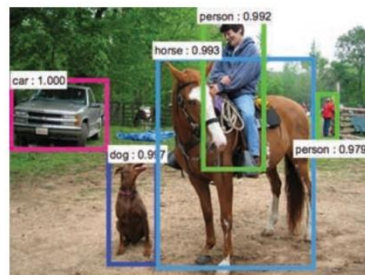
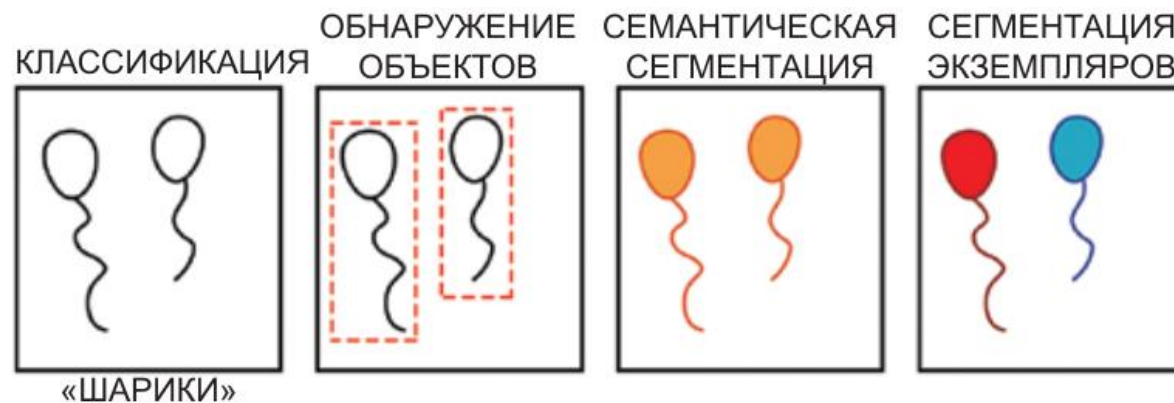
- LeNet-5;
- AlexNet;
- VGGNet и др.

Обнаружение объектов:

- R-CNN;
- Fast R-CNN;
- Faster R-CNN;
- YOLO и др.

Сегментация изображений:

- Mask R-CNN;
- U-Net и др.



Обнаружение объектов

При решении задачи обнаружения объектов модель выполняет следующие действия:

1. Выявление области, представляющей интерес.
2. Автоматическое извлечение признаков из этой области.
3. Классификация области.



R-CNN

Модель R-CNN была предложена в 2013 году Россом Гиршиком из Калифорнийского университета в Беркли. Принцип работы:

1. Выполняет выборочный поиск интересных областей (Regions Of Interest, ROI) на изображении.
2. Извлекает признаки из этих областей с помощью сверточной нейронной сети за два прохода.
3. Комбинирует линейную регрессию и метод опорных векторов для уточнения расположения ограничительных рамок и классификации объектов в каждой из этих рамок.

Модель R-CNN существенно повысила точность решения задачи обнаружения объектов.

Недостатки R-CNN:

- на вход можно было передать только одно изображение фиксированного размера;
- медленная скорость работы, высокие требования к вычислительным ресурсам.

Fast R-CNN

Россом Гиршик продолжил разработку и создал модель Fast R-CNN2 для повышения скорости работы модели. Принцип работы:

1. Выполняет выборочный поиск интересных областей (Regions Of Interest, ROI) на изображении.
2. Извлекает признаки из этих областей с помощью сверточной нейронной сети за один проход.
3. Использует полносвязанные слои для уточнения расположения ограничительных рамок и классификации объектов в каждой из этих рамок.

Fast R-CNN имеет более низкую вычислительную сложность и более высокую скорость работы.

Faster R-CNN

Модель Faster R-CNN была разработана в 2015 году Шаоцином Реном из Microsoft Research.

Для поиска ROI используются карты активаций из сверточных слоев.

Карта активации имеет два измерения и может рассматриваться как карта признаков для изображения.

Таким образом, Faster R-CNN использует сверточную сеть для выполнения всех трех шагов обнаружения объектов.

За счет этого значительно повышается скорость работы.

YOLO (You Only Look Once)

Модель YOLO предложена Джозефом Редмоном и его коллегами в 2015 году.

Принцип работы:

1. Извлекает признаки с помощью предварительно обученной сверточной сети.
2. Изображение делится на ряд ячеек, и для каждой прогнозируется количество ограничивающих рамок и вероятности классификации объектов.
3. Объединение множества ограничивающих рамок меньшего размера при условии достаточно высокой вероятности присутствия объекта какого-либо класса на изображении.

Алгоритм превзошел по скорости Faster R-CNN, но при этом мог обнаружить мелкие объекты на изображении.

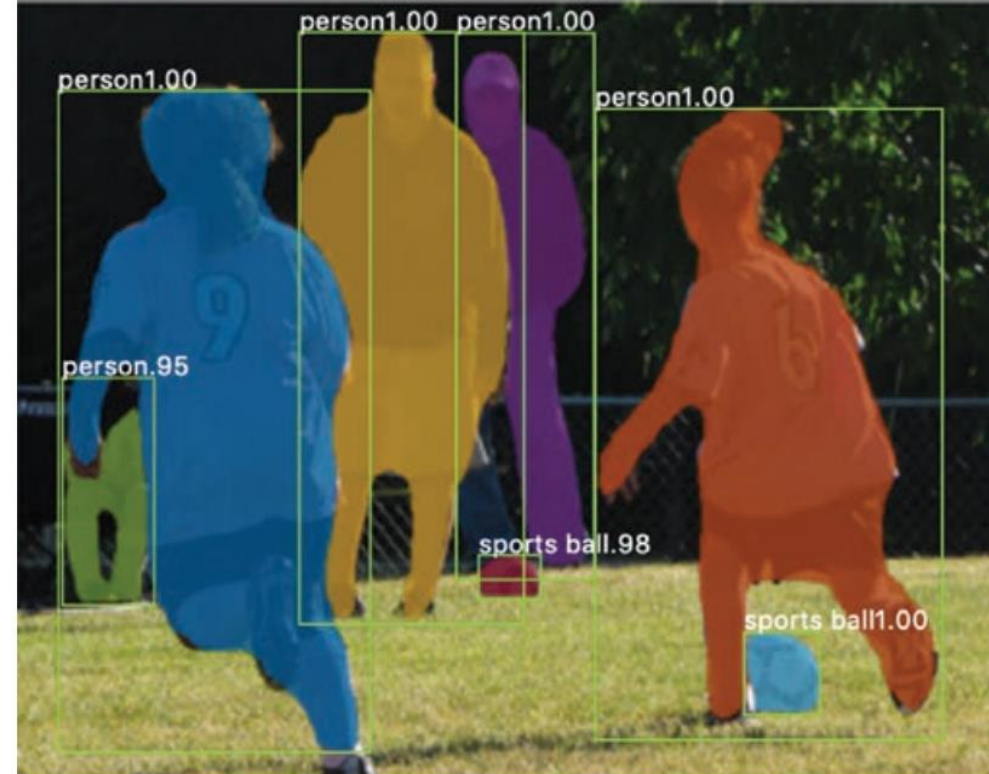
Сегментация изображений

Решение задачи сегментации изображений основано использовании бинарных масок в роли меток для обучения.

Бинарные маски представляют собой массивы тех же размеров, что и исходное изображение.

Бинарные маски вместо RGB-значений пикселов содержат нули и единицы, указывающие, где в изображении находится объект (1 – объект, 0 – остальное).

Для разных объектов на изображении формируется собственная бинарная маска.



Mask R-CNN

Алгоритм Mask R-CNN был разработан в Facebook AI Research (FAIR) в 2017 году.
Принцип работы:

1. Выявление интересных областей на изображении, которые могут содержать объекты, на основе архитектуры Faster R-CNN.
2. Классификация ROI для предсказания типа объекта в ограничивающей рамке и уточнения расположения и размера ограничивающей рамки.
3. Формирование ограничивающих рамок для получения частей карт признаков из сверточной сети, соответствующих данной части изображения.
4. Формирование маски, указывающей, какие пиксели соответствуют объекту на изображении, на основе карты признаков каждой интересной области (как входных данных для сверточной сети).

U-Net

Модель U-Net была создана в 2015 году в Университете Фрайберга с целью сегментации биомедицинских изображений. Принцип работы:

1. Использует сжимающий путь (contracting path) для создания последовательности меньших и более глубоких карт активаций, применяя несколько этапов свертки и объединения по максимальному значению. Сжимающий путь позволяет модели выявить мелкие признаки на изображении.
2. Использует разжимающий путь (expanding path) для восстановления карт активации до полного разрешения, выполняя несколько этапов повышения дискретизации и свертки. Разжимающий путь локализует мелкие признаки на изображении окончательных размеров.

Сжимающий и разжимающий пути симметричны (образуют U-образную форму), благодаря чему карты активаций, полученные в сжимающем пути, могут объединяться с полученными в разжимающем пути.

Передача данных из сжимающего пути в разжимающий последующий путь последующий сверточный слой позволяет сети научиться точно выявлять и локализовать признаки в двумерном пространстве.

Капсульные сети

В 2017 году Сара Сабур и ее коллеги из Google Brain в Торонто представили новую архитектуру ИНС под названием капсульные сети (capsule networks).

Капсульные сети, в отличие от сверточных, позволяют учитывать информацию о местоположении признаков, значительно повышает качество.

В настоящее время капсульные сети требуют значительных вычислительных ресурсов и поэтому редко используются на практике.



Глубокое обучение в NLP

Для применения глубокого обучения в NLP требуется выполнение операций предобработки текстов:

1. Токенизация.
2. Приведение к нижнему регистру.
3. Удаление стоп-слов.
4. Удаление знаков препинания.
5. Стемминг или лемматизация.
6. Выделение n-грамм.

Данные методы предобработки следует применять в зависимости от особенностей задачи и свойств данных.

- Стемминг может пригодиться при использовании небольших корпусов, но почти не имеет ценности для больших.
- Преобразование всех символов в нижний регистр может пригодиться при использовании небольших корпусов.
- Удаление знаков препинания обычно не дает преимуществ.
- В некоторых задачах не рекомендуется выполнять удаление стоп-слов (анализ тональности).

Чем больше корпус текстов, тем меньше методов предобработки следует использовать!

Векторное представление слов в NLP

При решении задач NLP методами глубокого обучения всегда следует использовать векторизацию текстов для получения векторного представления слов (эмбеддингов).

Для векторизации можно использовать:

- word2vec, doc2vec;
- FastText;
- BERT;
- функции библиотек или фреймворков и т. д.

Векторизация слов позволяет перейти от текстового представления слова к его идентификатору в векторном пространстве.

Векторное представление слов позволяет значительно сократить количество обрабатываемых данных, снизить затраты вычислительных ресурсов и повысить скорость обучения и вывода модели.

Особенности анализа текстов

Рекуррентные нейронные сети хорошо подходят для решения задачи NLP.

Давайте попробуем их использовать.

Сверточная нейронная сеть не сможет найти связь между «рекуррентными нейронными сетями» и словом «их».

Также сверточная нейронная сеть не сможет понять для чего рекуррентные сети необходимо использовать.

Человек при чтении текста запоминает контекст и постоянно возвращается к нему, для расшифровки соответствующих слов-ссылок.

Рекуррентная нейронная сеть работает аналогично.

Рекуррентные нейронные сети

Для анализа текстов возможно применение сверточных нейронных сетей, но сверточные сети выполняют анализ только на уровне n -грамм (n равен размеру фильтра).

Для анализа текстов лучше подойдут ИНС, которые могут учитывать длинные последовательности слов.

Рекуррентные нейронные сети (Recurrent Neural Network, RNN) – архитектура для обработки длинных последовательностей данных.

Семейство RNN включает специализированные типы слоев:

- ячейки долгой краткосрочной памяти (Long Short-Term Memory, LSTM)
- управляемые рекуррентные блоки (Gated Recurrent Unit, GRU).

Сети RNN, включая LSTM и GRU, хорошо подходят для обработки не только данных на естественном языке, но и любых других последовательных данных (временные ряды): стоимость ценных бумаг, финансовые показатели, периодические данные с датчиков и др.

Принцип работы RNN

Пунктирная линия означает петлю для передачи информации между шагами в RNN.

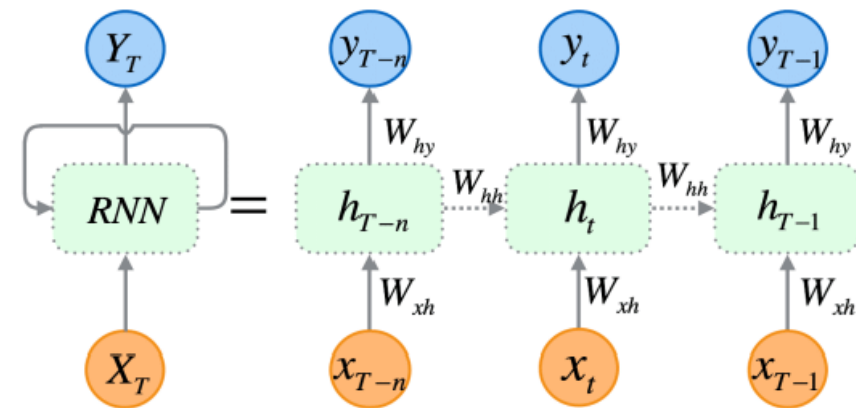
Под шагом понимается обработка отдельного элемента данных (слово, точка ряда).

Каждому слову в предложении соответствует свой RNN-нейрон (модуль).

Для каждого входа в RNN создается отдельный нейрон (как и в полносвязанном слое).

Поэтому при использовании RNN все предложения должны иметь одинаковую длину:

- длинные предложения необходимо обрезать;
- короткие предложения необходимо дополнять.



Особенности RNN

1. Каждый RNN-нейрон имеет дополнительный вход, связанный с предыдущим нейроном, благодаря чему сеть может передавать информацию из более ранних шагов последовательности данных.
2. Рекуррентные нейронные сети имеют более высокую вычислительную сложность на этапе обучения, по сравнению с полносвязанными и сверточными сетями (сети прямого распространения).
3. При обратном распространении в RNN стоимость распространяется не только от выходного слоя ко входному, но и обратно по шагам рекуррентного слоя (от позднего к раннему по хронологии).
4. Аналогично происходит затухание градиента: по слоям и по шагам.

LSTM

RNN рекомендуется использовать для данных, в которых расстояние между информацией и контекстом не превышает 10 шагов.

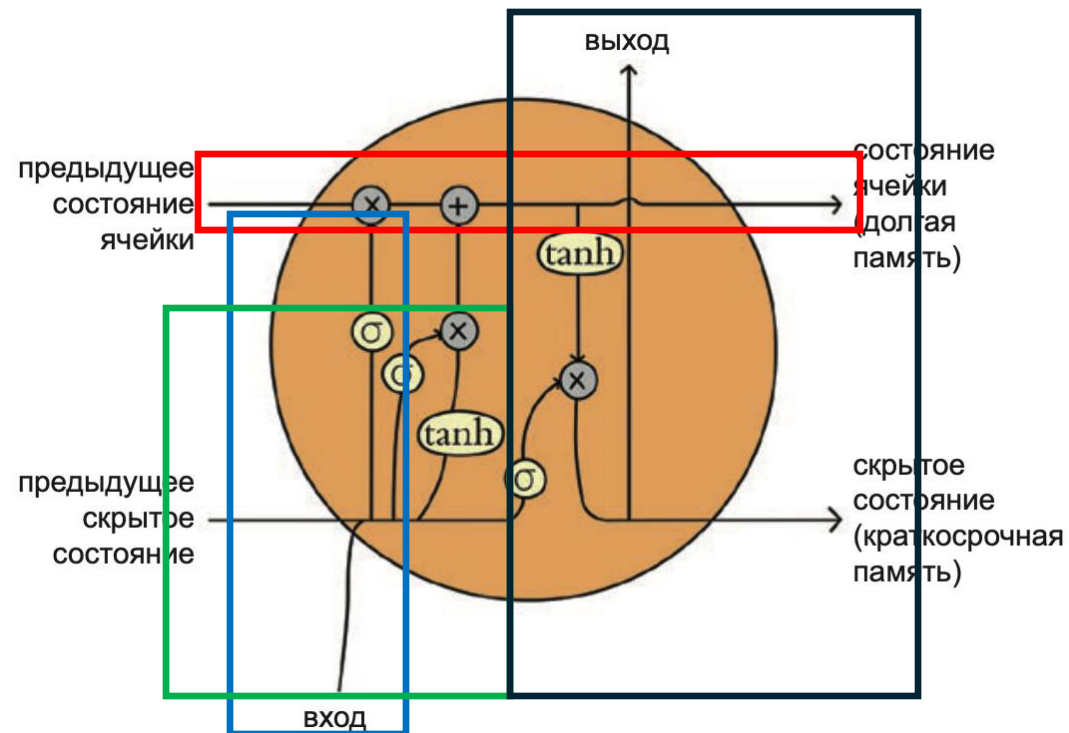
Если требуется учитывать более обширный контекст, то рекомендуется использовать LSTM-нейроны.

Архитектура LSTM была предложена в 1997 году Зеппом Хохрайтером и Юргеном Шмидхубером.

В настоящее время LSTM широко используется для решения различных задач в области NLP.

Принцип работы LSTM

1. Передача состояния (красный). Состояние текущего LSTM-нейрона (ячейки) передается в следующий нейрон практически без изменений с небольшими линейными преобразованиями (умножение и сложение).
2. Забывание (синий). Новая информация из текущего шага для добавления к текущему состоянию проходит через сигмоидную функцию активации (σ): 0 – удалить все, 1 – сохранить все.
3. Сохранение (зеленый). Новая информация на текущем шаге объединяется с скрытым состоянием предыдущего шага. σ определяет что обновлять, а $\tanh$ что добавить. Затем значения объединяются.
4. Фильтрация выхода (черный). σ определяет какие части состояния выводить, $\tanh$ нормирует значение в $[-1, 1]$. Затем значения перемножаются.

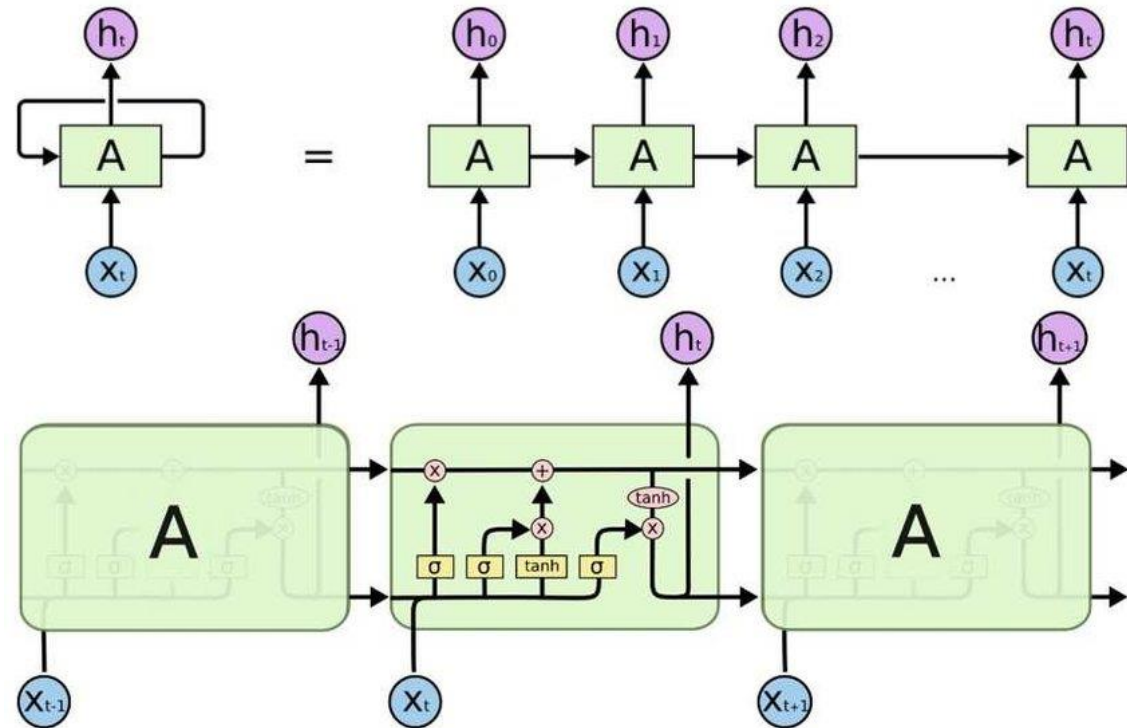
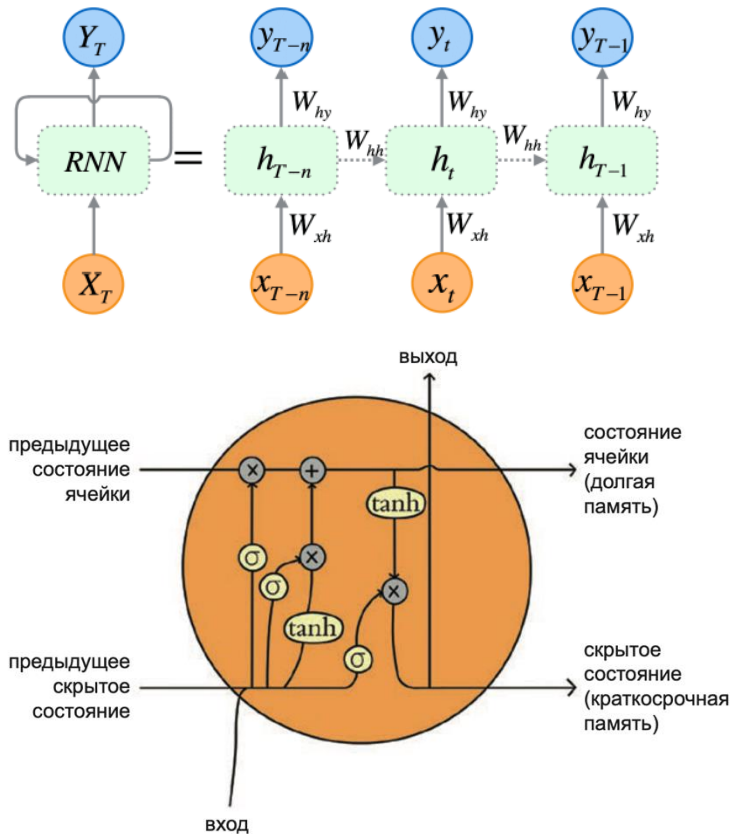


Особенности LSTM

1. Долгая память LSTM позволяет сохранять информацию в течение всех шагов.
2. Краткосрочная память работает аналогично RNN-нейронам.
3. Каждый LSTM-нейрон представляет собой отдельный элемент данных (шаг последовательности).
4. На каждом шаге принимается решение о соответствии информации данного шага локальному (краткосрочная память) и глобальному (долгосрочная память) контекстам.
5. Первые два вентиля (σ) определяют, относится ли информация из текущего временного шага к глобальному контексту и как она будет объединена в этот поток (умножение или сумма).
6. Конечный вентиль (σ) определяет, относится ли информация из текущего временного шага к локальному контексту.
7. Вентили σ определяют необходимость добавления информации в соответствующее состояние LSTM-нейрона (локальный и/или глобальный контекст).

Различия между RNN и LSTM

1. RNN ячейки передают между временными шагами информацию только одного типа (скрытое состояние) и содержат только одну функцию активации.
2. LSTM ячейки передают между временными шагами информацию двух типов: скрытое состояние и состояние ячейки и содержат пять функций активации.



Гиперпараметры RNN

1. Количество слоев. Более глубокие сети могут обнаружить более сложные признаки. Чем больше слоев, тем выше вычислительная сложность обучения. Рекомендуется начинать с 1-3 слоев.
2. Количество RNN-нейронов (модулей). Большее количество модулей позволяет обрабатывать больше информации, но также может привести к переобучению и повышению вычислительной сложности обучения. Рекомендуется начинать 128 нейронов и увеличивать их в 2 раза при необходимости.
3. Размер входного вектора: количество шагов и размер пространства признаков на каждом шаге. Рекомендуется от 10 до 100.
 - Если у вас много данных, то рекомендуется использовать более длинные последовательности.
 - Если данных мало, то короткие последовательности могут помочь предотвратить переобучение.
4. Тип RNN. Рекомендуется использовать LSTM (Long Short-Term Memory) или GRU (Gated Recurrent Unit) вместо стандартных RNN, поскольку они лучше справляются с долгосрочными зависимостями.
5. Прореживание. Рекомендуется значение от 0.2 до 0.5.

Трансформеры

Трансформер – вид нейросетевой архитектуры, который хорошо подходит для обработки последовательностей данных.

Трансформеры могут обрабатывать последовательности параллельно, что значительно ускоряет их обучение и вывод.

Трансформеры создают цифровое представление каждого элемента последовательности, инкапсулируют важную информацию о нём и окружающем его контексте.

Получившиеся представления затем можно передать в другие нейросети, которые воспользуются этой информацией для решения разных задач, в том числе для синтеза и классификации.

Generative pre-trained transformer или GPT (генеративный предобученный трансформер) — это тип нейронных языковых моделей, которые обучаются на больших наборах текстовых данных, чтобы генерировать текст, схожий с человеческим. Предобучение относится к начальному процессу обучения на корпусе, в результате которого модель учится предсказывать следующее слово в тексте и получает основу для успешного выполнения дальнейших задач, не имея больших объёмов данных.

LLM

Большая языковая модель (large language model, LLM) – языковая модель, состоящая из нейронной сети со множеством параметров (обычно миллиарды весовых коэффициентов и более), обученной на большом количестве неразмеченного текста с использованием обучения без учителя.

LLM хорошо справляются с широким спектром задач. Это сместило фокус исследований обработки естественного языка с предыдущей парадигмы обучения специализированных контролируемых моделей для конкретных задач.

Термин «большой» имеет два смысла:

1. Количество параметров. Параметры представляют собой переменные и веса, используемые моделью для формирования прогнозов при обработке языка. Размерность современных языковых моделей варьируется от десятков миллионов параметров до триллиона параметров и более.
2. Объем обучающих данных достигает нескольких петабайт и содержит триллионы токенов (базовых единиц текста или кода), обычно состоящих из нескольких символов, которые обрабатываются моделью.

GPT

Generative pre-trained transformer или GPT (генеративный предобученный трансформер) –тип нейронных языковых моделей, которые обучаются на больших наборах текстовых данных, чтобы генерировать текст, схожий с человеческим.

LLM могут основываться на различных архитектурах и методах обучения и используют разные подходы к представлению языка.

Предобучение относится к начальному процессу обучения на корпусе, в результате которого модель учится предсказывать следующее слово в тексте и получает основу для успешного выполнения дальнейших задач, не имея больших объемов данных.

GPT представляют собой наиболее мощные LLM, которые могут адаптироваться к конкретным задачам посредством дополнительного обучения или подсказок, обладают способностью прогнозировать синтаксические, семантические и онтологические структуры языка, одновременно наследуя неточности и предвзятости из обучающих данных.

Каждая GPT является LLM, но не каждая LLM является GPT.

LLM включает в себя широкий спектр моделей.

Механизм внимания

Механизм внимания (или attention mechanism) – возможность передачи полной последовательности скрытых состояний для всех шагов из одной части ИНС в другую.

Этот механизм позволяет моделям фокусироваться на наиболее релевантной информации, игнорируя менее важные данные.

Модель обращает внимание на наиболее релевантные элементы последовательности данных:

1. Механизм внимания взвешивает контексты, доступные для данного временного шага. Для этого каждое скрытое состояние умножается на его оценки softmax, что помогает усилить наиболее релевантные контексты (они будут иметь высокие оценки и, следовательно, более высокие вероятности softmax) и ослабить нерелевантные.
2. Взвешенные скрытые состояния суммируются, и новый вектор контекста используется для прогнозирования выхода на каждом последующем временном шаге части ИНС, которой были переданы скрытые состояния.

При таком подходе модель выборочно анализирует то, что ей известно о входной последовательности, и использует только релевантную информацию, где это необходимо, для формирования выхода.

Перенос обучения (transfer learning)

Для обучения глубоких моделей со сложной архитектурой требуется обширный и качественный набор данных, а также существенные вычислительные ресурсы и временные затраты.

После обучения глубокие нейронные сети содержат коллекцию различных признаков, комбинируя которые можно решить практически любую задачу.

Принцип переноса обучения позволяет использовать коллекцию признаков обученных глубоких моделей для решения новой задачи путем обучения только выходных полносвязанных слоев (в большинстве случаев).

Для компьютерного зрения и NLP можно использовать обширную коллекцию доступных моделей:

1. Для компьютерного зрения: VGG, ResNet, и др.
2. Для NLP: ULMFiT, ELMo, BERT, и др.

Выводы

1. Подбирайте архитектуру под задачу.
2. Подбирайте оптимальные гиперпараметры.
3. Используйте перенос обучения.
4. В настоящее время обучить свою глубокую ИНС может каждый.
5. НО **только решение большого числа задач** позволит грамотно выбирать архитектуру и подбирать оптимальные гиперпараметры.